MODUL PRAKTIKUM PEMROGRAMAN WEB

MODUL 7: MEMBUAT FITUR CRUD DENGAN JAVASCRIPT & LARAGON

Implementasi Operasi Create, Read, Update, Delete Berbasis 100% JavaScript (Node.js & Fetch API) Terhubung ke MySQL Laragon

BAB 1: ARSITEKTUR CRUD DENGAN JAVASCRIPT & MYSQL LARAGON

1.1 Arsitektur Aplikasi Web Modern

Pada modul-modul sebelumnya, kita telah mempelajari manipulasi antarmuka web (Modul 5) dan pertukaran data asinkronus menggunakan Fetch API (Modul 6). Kini, seluruh alur aplikasi web dibangun 100% menggunakan bahasa JavaScript:

- **Frontend (Client-Side JS):** Menggunakan Vanilla JavaScript di peramban untuk menangani formulir pengguna, mencegah reload halaman (e.preventDefault()), mengirim permintaan HTTP asinkronus (fetch() dengan async/await), dan me-render hasil data ke tabel DOM secara instan.
- **Backend (Server-Side JS):** Menggunakan Node.js dan Express.js (server.js) untuk melayani rute REST API, memproses data JSON, dan berkomunikasi dengan server database.
- **Database Server (MySQL di Laragon):** Laragon menjalankan layanan MySQL Database Server pada port 3306 untuk menyimpan rekaman data secara permanen.

1.2 Menyiapkan Node.js di Laragon

Laragon merupakan lingkungan universal yang telah dilengkapi Node.js dan NPM. Untuk memastikan Node.js telah aktif:

1. Buka aplikasi Terminal/Command Promt). Ketik perintah: node -v dan npm -v untuk memeriksa versi Node.js yang terpasang.

2. Pastikan tombol 'Start All' pada Laragon sudah diklik sehingga MySQL Server berstatus running.

1.3 Membuat Database & Tabel MySQL di Laragon

Sebelum menulis kode program, kita menyiapkan basis data db_kampus dan tabel mahasiswa melalui pengelola database bawaan Laragon (HeidiSQL atau phpMyAdmin).

- **Atribut Kolom Tabel Mahasiswa:** id (Primary Key, Auto Increment), nim (Nomor Induk Mahasiswa, unik), nama (Nama Lengkap), jurusan (Program Studi), dan email.

Jalankan perintah SQL DDL berikut di HeidiSQL atau phpMyAdmin:

```
-- 1. Membuat Database Baru
CREATE DATABASE IF NOT EXISTS db_kampus;
USE db_kampus;
```

```
-- 2. Membuat Tabel Mahasiswa
CREATE TABLE IF NOT EXISTS mahasiswa (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nim VARCHAR(20) NOT NULL UNIQUE,
    nama VARCHAR(100) NOT NULL,
    jurusan VARCHAR(50) NOT NULL,
    email VARCHAR(100) NOT NULL
);

-- 3. Memasukkan Data Awal Sederhana
INSERT INTO mahasiswa (nim, nama, jurusan, email) VALUES
('240101001', 'Ahmad Pratama', 'Teknik Informatika', 'ahmad@kampus.ac.id'),
('240101002', 'Siti Nurhaliza', 'Sistem Informasi', 'siti@kampus.ac.id');
```

1.4 Membangun Koneksi Database MySQL Menggunakan JavaScript

Di lingkungan Node.js, koneksi ke MySQL dilakukan menggunakan paket resmi mysql2. Objek koneksi dibuat menggunakan parameter host: 'localhost', user: 'root', password: '' (kosong bawaan Laragon), dan database: 'db_kampus'.

BAB 2: PANDUAN PRAKTIK TERSTRUKTUR (SISTEM CRUD MAHASISWA)

Buka terminal Laragon, lalu arahkan ke folder proyek Anda di C:\laragon\www\crud-mahasiswa\. Struktur proyek terdiri dari berkas backend (server.js, package.json) dan folder frontend statis (public/index.html, public/style.css, public/script.js).

Langkah 2.1: Inisialisasi Proyek Node.js & Instalasi Paket

Jalankan perintah berikut pada terminal di folder C:\laragon\www\crud-mahasiswa\:

```
npm init -y
npm install express mysql2
```

Perintah di atas akan menghasilkan berkas package.json yang mencatat dependensi framework Express dan driver MySQL.

Langkah 2.2: Membangun Server Backend REST API (server.js)

Buat berkas server.js di folder root proyek. Berkas ini bertindak sebagai server HTTP yang menghubungkan JavaScript ke database MySQL di Laragon:

```
// File: server.js - Backend Node.js Express & MySQL
const express = require('express');
const mysql = require('mysql2');
const app = express();
const PORT = 3000;
```

```
// Middleware untuk parsing JSON dan menyajikan file statis frontend
app.use(express.json());
app.use(express.static('public'));

// 1. Koneksi ke Database MySQL Laragon
const db = mysql.createConnection({
    host: 'localhost',
    user: 'root',
    password: '', // Password default MySQL Laragon kosong
    database: 'db_kampus'
});

db.connect((err) => {
    if (err) {
        console.error('Koneksi database gagal:', err.message);
        return;
    }
    console.log('Terhubung ke database MySQL Laragon');
});

// 2. READ: Ambil seluruh data mahasiswa [Indikator 5.3, 6.2]
app.get('/api/mahasiswa', (req, res) => {
    db.query('SELECT * FROM mahasiswa ORDER BY id DESC', (err, results) => {
        if (err) return res.status(500).json({ success: false, message: err.message });
        res.json(results);
    });
});

// 3. CREATE: Tambah mahasiswa baru
app.post('/api/mahasiswa', (req, res) => {
    const { nim, nama, jurusan, email } = req.body;
    const sql = 'INSERT INTO mahasiswa (nim, nama, jurusan, email) VALUES (?, ?, ?,
?)';
    db.query(sql, [nim, nama, jurusan, email], (err, result) => {
        if (err) return res.status(500).json({ success: false, message: err.message });
        res.json({ success: true, message: 'Data mahasiswa berhasil disimpan!' });
    });
});

// 4. UPDATE: Ubah data mahasiswa
app.put('/api/mahasiswa/:id', (req, res) => {
    const { id } = req.params;
    const { nim, nama, jurusan, email } = req.body;
    const sql = 'UPDATE mahasiswa SET nim = ?, nama = ?, jurusan = ?, email = ? WHERE
id = ?';
    db.query(sql, [nim, nama, jurusan, email, id], (err, result) => {
        if (err) return res.status(500).json({ success: false, message: err.message });
        res.json({ success: true, message: 'Data mahasiswa berhasil diperbarui!' });
    });
});

// 5. DELETE: Hapus data mahasiswa
```

```
app.delete('/api/mahasiswa/:id', (req, res) => {
    const { id } = req.params;
    db.query('DELETE FROM mahasiswa WHERE id = ?', [id], (err, result) => {
        if (err) return res.status(500).json({ success: false, message: err.message });
        res.json({ success: true, message: 'Data mahasiswa berhasil dihapus!' });
    });
});

app.listen(PORT, () => {
    console.log(`Server CRUD aktif di http://localhost:${PORT}`);
});
```

Langkah 2.3: Membuat Antarmuka Web (public/index.html & public/style.css)

Buat folder bernama public di dalam folder proyek. Di dalam folder public, buat berkas index.html berikut:

```
<!DOCTYPE html>
<html lang="id">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>CRUD Mahasiswa - JavaScript & Laragon</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <div class="container">
        <h2>Manajemen Data Mahasiswa</h2>
        <p class="subtitle">Sistem CRUD 100% JavaScript: Node.js, Express, Fetch API &
MySQL Laragon</p>

        <!-- Pesan Notifikasi Alert -->
        <div id="alertMessage" class="alert" style="display:none;"></div>

        <!-- Formulir Tambah / Edit Mahasiswa -->
        <div class="form-card">
            <h3 id="formTitle">Tambah Mahasiswa Baru</h3>
            <form id="mhsForm">
                <input type="hidden" id="mhsId">
                <div class="form-row">
                    <div class="form-group flex-1">
                        <label for="nim">NIM <span class="req">*</span></label>
                        <input type="text" id="nim" required placeholder="Contoh:
240101003">
                    </div>
                    <div class="form-group flex-1">
                        <label for="nama">Nama Lengkap <span
class="req">*</span></label>
                        <input type="text" id="nama" required placeholder="Nama
mahasiswa">
                    </div>
                </div>
```

```
                <div class="form-row">
                    <div class="form-group flex-1">
                        <label for="jurusan">Program Studi <span
class="req">*</span></label>
                        <select id="jurusan" required>
                            <option value="">-- Pilih Jurusan --</option>
                            <option value="Teknik Informatika">Teknik
Informatika</option>
                            <option value="Sistem Informasi">Sistem Informasi</option>
                            <option value="Manajemen Informatika">Manajemen
Informatika</option>
                        </select>
                    </div>
                    <div class="form-group flex-1">
                        <label for="email">Email <span class="req">*</span></label>
                        <input type="email" id="email" required
placeholder="nama@kampus.ac.id">
                    </div>
                </div>
                <div class="form-actions">
                    <button type="submit" id="btnSubmit" class="btn btn-primary">Simpan
Mahasiswa</button>
                    <button type="button" id="btnCancel" class="btn btn-secondary"
style="display:none;">Batal Edit</button>
                </div>
            </form>
        </div>

        <!-- Tabel Data Mahasiswa [Indikator 6.3] -->
        <div class="table-card">
            <table>
                <thead>
                    <tr>
                        <th>No</th>
                        <th>NIM</th>
                        <th>Nama Lengkap</th>
                        <th>Jurusan</th>
                        <th>Email</th>
                        <th>Aksi</th>
                    </tr>
                </thead>
                <tbody id="mhsTableBody"></tbody>
            </table>
        </div>
    </div>

    <script src="script.js"></script>
</body>
</html>
```

Selanjutnya, buat berkas style.css di dalam folder public:

```
/* File: public/style.css */
* { box-sizing: border-box; margin: 0; padding: 0; font-family: 'Segoe UI', Arial,
sans-serif; }
body { background-color: #f1f5f9; color: #0f172a; padding: 30px 15px; }
.container { max-width: 900px; margin: 0 auto; }
h2 { color: #1e3a8a; margin-bottom: 4px; font-size: 22px; }
.subtitle { color: #64748b; font-size: 13.5px; margin-bottom: 20px; }
.form-card, .table-card { background: #ffffff; padding: 22px; border-radius: 10px;
box-shadow: 0 4px 6px rgba(0,0,0,0.04); margin-bottom: 20px; border: 1px solid #e2e8f0;
}
.form-card h3 { color: #1e3a8a; font-size: 16px; margin-bottom: 14px; border-bottom:
2px solid #e2e8f0; padding-bottom: 8px; }
.form-row { display: flex; gap: 14px; margin-bottom: 12px; }
.flex-1 { flex: 1; }
.form-group label { display: block; font-size: 12.5px; font-weight: 600; margin-bottom:
4px; color: #334155; }
.req { color: #dc2626; }
input[type="text"], input[type="email"], select {
    width: 100%; padding: 9px 12px; border: 1px solid #cbd5e1; border-radius: 6px;
font-size: 13.5px; outline: none;
}
input:focus, select:focus { border-color: #1e3a8a; }
.form-actions { display: flex; gap: 8px; margin-top: 10px; }
.btn { padding: 8px 16px; border: none; border-radius: 6px; font-weight: 600;
font-size: 13px; cursor: pointer; transition: 0.2s; }
.btn-primary { background-color: #1e3a8a; color: white; }
.btn-primary:hover { background-color: #1e40af; }
.btn-secondary { background-color: #64748b; color: white; }
.btn-action { padding: 5px 10px; border-radius: 4px; font-size: 12px; font-weight:
bold; cursor: pointer; border: none; }
.btn-edit { background-color: #0284c7; color: white; margin-right: 4px; }
.btn-delete { background-color: #dc2626; color: white; }
table { width: 100%; border-collapse: collapse; }
th, td { padding: 11px 12px; text-align: left; border-bottom: 1px solid #e2e8f0;
font-size: 13.5px; }
th { background-color: #1e3a8a; color: white; font-weight: 600; }
tr:hover { background-color: #f8fafc; }
.alert { padding: 10px 14px; border-radius: 6px; margin-bottom: 16px; font-size: 13px;
font-weight: 500; }
.alert-success { background-color: #dcfce7; color: #166534; border: 1px solid #bbf7d0;
}
.alert-error { background-color: #fee2e2; color: #991b1b; border: 1px solid #fecaca; }
```

Langkah 2.4: Mengembangkan Logika Frontend JavaScript (public/script.js) [Indikator 5.3, 6.1, 6.2, 6.3]

Buat berkas script.js di dalam folder public. Kode ini menggunakan Fetch API dengan async/await untuk berkomunikasi secara asinkronus ke server backend Node.js tanpa me-reload peramban:

```
// File: public/script.js - Logika Client-Side JavaScript CRUD
document.addEventListener("DOMContentLoaded", () => {
    // 1. Tangkap Elemen DOM
```

```
    const mhsForm = document.querySelector("#mhsForm");
    const mhsIdInput = document.querySelector("#mhsId");
    const nimInput = document.querySelector("#nim");
    const namaInput = document.querySelector("#nama");
    const jurusanInput = document.querySelector("#jurusan");
    const emailInput = document.querySelector("#email");
    const btnSubmit = document.querySelector("#btnSubmit");
    const btnCancel = document.querySelector("#btnCancel");
    const formTitle = document.querySelector("#formTitle");
    const mhsTableBody = document.querySelector("#mhsTableBody");
    const alertMessage = document.querySelector("#alertMessage");

    let listMahasiswa = [];

    // Fungsi Pembantu Notifikasi Alert
    const showAlert = (text, type = "success") => {
        alertMessage.textContent = text;
        alertMessage.className = `alert alert-${type}`;
        alertMessage.style.display = "block";
        setTimeout(() => { alertMessage.style.display = "none"; }, 3000);
    };

    // 2. READ: Ambil Data Mahasiswa dari API Node.js
    const loadMahasiswa = async () => {
        try {
            const res = await fetch("/api/mahasiswa");
            listMahasiswa = await res.json();
            renderTable(listMahasiswa);
        } catch (err) {
            showAlert("Gagal memuat data dari server backend", "error");
        }
    };

    const renderTable = (data) => {
        mhsTableBody.innerHTML = "";
        if (data.length === 0) {
            mhsTableBody.innerHTML = "<tr><td colspan='6' style='text-align:center;
color:#64748b;'>Belum ada data mahasiswa tersimpan.</td></tr>";
            return;
        }
        data.forEach((mhs, idx) => {
            const tr = document.createElement("tr");
            tr.innerHTML = `
                <td>${idx + 1}</td>
                <td>${mhs.nim}</td>
                <td>${mhs.nama}</td>
                <td>${mhs.jurusan}</td>
                <td>${mhs.email}</td>
                <td>
                    <button class="btn-action btn-edit"
data-id="${mhs.id}">Edit</button>
                    <button class="btn-action btn-delete"
data-id="${mhs.id}">Hapus</button>
```

```
            </td>
        `;
        mhsTableBody.appendChild(tr);
    });
};

// 3. CREATE & UPDATE: Submit Formulir
mhsForm.addEventListener("submit", async (e) => {
    e.preventDefault();

    const payload = {
        nim: nimInput.value.trim(),
        nama: namaInput.value.trim(),
        jurusan: jurusanInput.value,
        email: emailInput.value.trim()
    };

    const id = mhsIdInput.value;
    const isEdit = Boolean(id);
    const url = isEdit ? `/api/mahasiswa/${id}` : "/api/mahasiswa";
    const method = isEdit ? "PUT" : "POST";

    try {
        const res = await fetch(url, {
            method: method,
            headers: { "Content-Type": "application/json" },
            body: JSON.stringify(payload)
        });
        const result = await res.json();

        if (result.success) {
            showAlert(result.message);
            resetForm();
            loadMahasiswa();
        } else {
            showAlert("Terjadi kesalahan: " + result.message, "error");
        }
    } catch (err) {
        showAlert("Gagal mengirim data ke server", "error");
    }
});

// 4. DELETE & EDIT: Tangani Event Klik pada Tabel (Event Delegation)
mhsTableBody.addEventListener("click", async (e) => {
    const id = Number(e.target.dataset.id);
    if (!id) return;

    // Mode Edit: Isi form otomatis
    if (e.target.classList.contains("btn-edit")) {
        const targetMhs = listMahasiswa.find(m => m.id === id);
        if (targetMhs) {
            mhsIdInput.value = targetMhs.id;
            nimInput.value = targetMhs.nim;
```

```
                namaInput.value = targetMhs.nama;
                jurusanInput.value = targetMhs.jurusan;
                emailInput.value = targetMhs.email;

                formTitle.textContent = "Edit Data Mahasiswa";
                btnSubmit.textContent = "Perbarui Mahasiswa";
                btnCancel.style.display = "inline-block";
                window.scrollTo({ top: 0, behavior: "smooth" });
            }
        }
        // Mode Delete: Hapus data via HTTP DELETE
        else if (e.target.classList.contains("btn-delete")) {
            if (confirm("Apakah Anda yakin ingin menghapus data mahasiswa ini?")) {
                try {
                    const res = await fetch(`/api/mahasiswa/${id}`, { method: "DELETE"
});
                    const result = await res.json();
                    if (result.success) {
                        showAlert(result.message);
                        loadMahasiswa();
                    }
                } catch (err) {
                    showAlert("Gagal menghapus data dari server", "error");
                }
            }
        }
    });

    // 5. Reset Form ke Mode Tambah
    const resetForm = () => {
        mhsForm.reset();
        mhsIdInput.value = "";
        formTitle.textContent = "Tambah Mahasiswa Baru";
        btnSubmit.textContent = "Simpan Mahasiswa";
        btnCancel.style.display = "none";
    };
    btnCancel.addEventListener("click", resetForm);

    // Ambil data pertama kali
    loadMahasiswa();
});
```

Langkah 2.5: Menjalankan Aplikasi & Uji Coba di Browser

1. Buka terminal Laragon di folder proyek, lalu jalankan server Node.js:

```
node server.js
```

2. Buka browser dan ketik alamat: http://localhost:3000.

3. Uji Operasi Read: Data mahasiswa awal otomatis ditarik dari MySQL dan muncul di tabel

4. Uji Operasi Create: Isi form, klik Simpan, dan pastikan data langsung bertambah ke database tanpa reload.

5. Uji Operasi Update: Klik tombol Edit, ubah data, klik Perbarui, dan amati perubahan data di tabel.

6. Uji Operasi Delete: Klik tombol Hapus, setujui dialog konfirmasi, dan pastikan baris data terhapus.

7. Simpan pekerjaan ke Git dengan perintah terminal (push ke branch baru misal pertemuan-7):

```
git add .
git commit -m "feat: menyelesaikan modul 7 crud fullstack javascript mysql laragon"
git push -u origin pertemuan-7
```

BAB 3: TUGAS MANDIRI & RUBRIK PENILAIAN

Kembangkan aplikasi Full-Stack JavaScript CRUD di atas dengan menambahkan 3 (tiga) fitur interaktif sederhana menggunakan JavaScript:

- **Fitur 1 - Kolom Pencarian Real-Time (Search Filter di JavaScript):** Tambahkan satu kolom input pencarian teks di atas tabel (<input type="text" id="searchInput" placeholder="Cari nama atau NIM...">). Pasang event listener 'input' pada JavaScript agar setiap kali pengguna mengetik huruf, tabel langsung tersaring secara instan menggunakan listMahasiswa.filter(...) persis seperti pada Modul 6.
- **Fitur 2 - Validasi Pencegahan NIM Duplikat di Frontend JavaScript:** Sebelum data dikirim via fetch() ke backend, lakukan validasi di JavaScript menggunakan method .some(): periksa apakah NIM yang diketikkan sudah ada di dalam listMahasiswa. Jika sudah ada, batalkan submit dan tampilkan pesan alert: 'NIM sudah terdaftar!'.
- **Fitur 3 - Indikator Counter Total Mahasiswa:** Tambahkan teks kecil di atas tabel: 'Total Mahasiswa Terdaftar: X Orang' yang nilainya otomatis diperbarui oleh JavaScript setiap kali data berhasil ditambah atau dihapus.

Setelah selesai, lakukan commit dan push ke repositori GitHub Anda:

```
git add .
git commit -m "feat: menyelesaikan tugas modul 7 crud fullstack javascript mysql
laragon"
git push -u origin pertemuan-7
```